

# A brief introduction to Git, GitHub, and Google Colab

Stefano Diciotti<sup>1</sup>

<sup>1</sup>Dipartimento di Ingegneria dell'Energia Elettrica e dell'Informazione "Guglielmo Marconi" - DEI, Università di Bologna

## Core tools

- **Python** – our main programming language
- **Git** – for managing source code locally and remotely
- **GitHub** – for collaboration and version control
- **Google Colab** – for cloud-based interactive development



## Git is a version control system

Git tracks changes to files over time, allowing multiple people to collaborate on the same codebase without overwriting each other's work.

### Why is it useful?

- You can experiment safely without losing previous versions
- You can see exactly who changed what, when, and why
- You can merge everyone's contributions into a shared project
- You can go back in time if something breaks

## Typical Git-based collaboration:

- Create a **branch** from the main project
- Make your own edits safely in that branch
- Use Git to **merge** your changes back into the main version
- Git keeps track of everyone's changes to avoid conflicts

**Result:** Everyone works on the same project, without overwriting each other's code.

## How these tools fit together:

- **Git** tracks code changes locally (your version control system)
- **GitHub** hosts your Git repositories online (where you collaborate)
- **Google Colab** runs your code (your execution environment)

For this course, we'll:

- Use **GitHub** to download shared code and notebooks
- Run them in **Google Colab** – no installation required

## GitHub is a cloud-based platform for code collaboration

GitHub allows you to store, manage, and share your code online. It is built on top of Git, and makes version control and teamwork easier and more accessible.

### Why use GitHub?

- Share and showcase your code with others
- Track and manage changes over time
- Work with others on the same project safely
- Review and improve each other's code

## Repositories are the core of GitHub

A **repository** (or *repo*) is a space where your project lives: it contains all the code, data, and history.

### In a GitHub repository, you can:

- Organize code and documentation in folders
- View the full history of changes
- Invite others to collaborate
- Open issues and propose improvements
- Access the code from tools like Google Colab

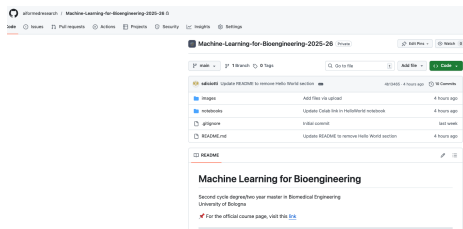
*Repositories can be public or private, depending on your needs.*

# Example of a GitHub Repository

## A typical GitHub repository contains:

- `README.md` – project description
- `.ipynb` files – notebooks for Colab
- `src/` – source code (Python files)
- `data/` – input datasets (if included)
- Version history and commit log

You can open a notebook from GitHub directly in Google Colab, run the code, and explore the project without installing anything.





## Google Colab

*Free cloud-based platform for AI development by Google*

<https://colab.research.google.com/>

### Specifications:

- **CPU** and **NVIDIA T4 GPU**
- **12.72 GB** RAM
- **112.64 GB** storage
- Pre-installed libraries: Keras, TensorFlow, PyTorch, OpenCV

## Google Colab

*A hosted Jupyter Notebook service by Google*

Colab is an online platform that allows you to write and execute Python code directly in your browser, with no setup required. It provides free access to computing resources such as **GPUs** and **TPUs**, making it particularly suitable for:

- Machine learning experiments
- Data science projects
- Educational activities

**Requirements:** You only need a Google account to get started.

### More info:

<https://research.google.com/colaboratory/faq.html>

# What is a Notebook?

## Jupyter Notebook (or Google Colab)

A **notebook** is an interactive environment that combines *code*, *text*, *formulas*, and *visualizations* in a single document. It is especially useful for:

- Prototyping and experimenting with code
- Exploring and visualizing data
- Writing computational narratives (mixing explanation and execution)
- Teaching and collaborative research

Notebooks execute code in **cells**, allowing for step-by-step development and immediate feedback.

## Notebooks are made of cells

Each notebook consists of a sequence of **cells**, which can be of two main types:

- **Code cells** – where you write and run code (e.g., Python)
- **Text cells (Markdown)** – where you can write formatted text, explanations, and even math formulas

### Examples:

- **Code cell:** `print("Hello, world!")`
- **Markdown cell (text):** This is a `\textbf{bold}` explanation.
- **Markdown cell (formula):** `$$ E = mc^2 $$` (rendered as:  $E = mc^2$ )

Cells can be run independently, allowing iterative and interactive development.

## Traditional Scripts

### Advantages:

- Better suited for production environments
- Easy to version control (e.g., with Git)
- Cleaner file structure for large projects

### Disadvantages:

- Less interactive
- Slower for prototyping
- No built-in visual feedback

## Notebooks (e.g., Colab)

### Advantages:

- Interactive and beginner-friendly
- Ideal for data exploration and visualization
- Combine code, text, and results

### Disadvantages:

- Harder to maintain for large-scale code
- Version control can be messy
- Not ideal for deployment

## Machine learning for age prediction

This demo shows how to train and evaluate a simple machine learning model to estimate a child's age from brain MRI structural images. It can be run on Google Colab or locally with minimal setup.

### Key features:

- Uses a preprocessed version of the NKI2 dataset (sample included)
- Based on Scikit-learn with simple machine learning models
- Includes code for training and testing

### Run the demo:

<https://github.com/sdiciotti/Age-Prediction-Demo>

- **Google Colaboratory FAQ:**  
<https://research.google.com/colaboratory/faq.html>
- **GitHub Git – Getting Started Guide:**  
<https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>
- **Age Prediction Demo:**  
<https://github.com/sdiciotti/Age-Prediction-Demo>